Name : Esona

Surname : Khosa

Student number : ST10544616

Subject : Programming

Subject code : PROG5121

Part 2 – Sending Messages

1. Message.java

```
import java.io.FileWriter;
import java.io.IOException;
import java.security.SecureRandom;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

class Message {

    private static final SecureRandom RANDOM = new SecureRandom();
    private static final int ID_LENGTH = 10;

    private String messageID;
    private String recipient;
    private String messageText;
    private int messageNumber;
    private String messageHash;

    private static final AtomicInteger totalMessagesSent = new
AtomicInteger(0);

    private static final List<Message> sentMessages =
            Collections.synchronizedList(new ArrayList<>());

    private static final List<Message> allMessages =
            Collections.synchronizedList(new ArrayList<>());

    public Message(String recipient, String messageText, int messageNumber) {
        this.recipient = recipient;
        this.messageText = messageText;
        this.messageNumber = messageNumber;

        this.messageID = generateMessageID();
        this.messageHash = createMessageHash();

        allMessages.add(this);
    }

    private String generateMessageID() {
        StringBuilder sb = new StringBuilder(ID_LENGTH);
        for (int i = 0; i < ID_LENGTH; i++) {
            int digit = RANDOM.nextInt(10);
```

```
            sb.append(digit);
        }
        return sb.toString();
    }

    public boolean checkMessageID() {
        return messageID != null && messageID.matches("\\d{" + ID_LENGTH +
"}");
    }

    /**
     * Validates recipient using an E.164-like format: a leading '+' followed
by 1..15 digits.
     * Returns a user-facing message for compatibility with existing code.
     */
    public String checkRecipientCell() {
        if (recipient != null && recipient.matches("^\\+\\d{1,15}$")) {
            return "Cell phone number successfully captured.";
        }
        return "Cell phone number is incorrectly formatted or does not contain
an international code. Please correct the number and try again.";
    }

    /**
     * Validates message text. Keeps original string-return API for
compatibility.
     */
    public String validateMessage() {
        if (messageText == null || messageText.trim().isEmpty()) {
            return "Please enter a message.";
        }

        int length = messageText.length();
        if (length <= 250) {
            return "Message ready to send.";
        }

        int excessCharacters = length - 250;
        return "Message exceeds 250 characters by " + excessCharacters + ",
please reduce the size.";
    }

    public String createMessageHash() {
        if (!checkMessageID() || messageText == null ||
messageText.trim().isEmpty()) {
            return "";
        }
```

```
        String[] words = messageText.trim().split("\\s+");
        String firstWord = "";
        String lastWord = "";

        if (words.length > 0) {
            firstWord = words[0].replaceAll("[^A-Za-z0-9]", "");
            lastWord = words[words.length - 1].replaceAll("[^A-Za-z0-9]", "");
        }

        String idPrefix = messageID.length() >= 2 ? messageID.substring(0, 2)
: messageID;
        return idPrefix + ":" + messageNumber + ":" + (firstWord +
lastWord).toUpperCase();
    }

    /**
     * Process user choice.
     * - choice == 1 : send the message (validates recipient & message)
     * - choice == 2 : delete the message from sentMessages (if present)
     * - choice == 3 : store the message to messages.json (append as NDJSON)
     *
     * Kept name SentMessage() for backward compatibility with original code,
but behavior fixed.
     */
    public String SentMessage(int choice) {
        if (choice == 1) {
            String v = validateMessage();
            if (!"Message ready to send.".equals(v)) {
                return v;
            }

            String r = checkRecipientCell();
            if (!"Cell phone number successfully captured.".equals(r)) {
                return r;
            }

            sentMessages.add(this);
            totalMessagesSent.incrementAndGet();

            System.out.println("Message ID: " + messageID);
            System.out.println("Message Hash: " + messageHash);
            System.out.println("Recipient: " + recipient);
            System.out.println("Message: " + messageText);

            return "Message successfully sent.";

        } else if (choice == 2) {
            boolean removed = sentMessages.remove(this);
```

```
            if (removed) {
                // ensure counter does not go negative
                int prev;
                do {
                    prev = totalMessagesSent.get();
                    if (prev <= 0) break;
                } while (!totalMessagesSent.compareAndSet(prev, Math.max(0,
prev - 1)));

                return "Message deleted from sent messages.";
            } else {
                return "Message not found in sent messages.";
            }

        } else if (choice == 3) {
            String v = validateMessage();
            if (!"Message ready to send.".equals(v)) {
                return v;
            }

            String r = checkRecipientCell();
            if (!"Cell phone number successfully captured.".equals(r)) {
                return r;
            }

            if (storeMessage()) {
                return "Message successfully stored.";
            }
            return "Message could not be stored.";

        } else {
            return "Invalid option selected.";
        }
    }

    public String printMessages() {
        StringBuilder output = new StringBuilder();

        synchronized (sentMessages) {
            for (Message message : sentMessages) {
                output.append("Message ID:
").append(message.messageID).append("\n");
                output.append("Message Hash:
").append(message.messageHash).append("\n");
                output.append("Recipient:
").append(message.recipient).append("\n");
                output.append("Message:
").append(message.messageText).append("\n\n");
```

```
            }
        }

        if (output.length() == 0) {
            return "No messages have been sent.";
        }

        return output.toString();
    }

    public int returnTotalMessages() {
        return totalMessagesSent.get();
    }

    /**
     * Appends a single JSON object (one-per-line, NDJSON) to messages.json.
     */
    public boolean storeMessage() {
        // Synchronized to avoid concurrent file appends mixing content
        synchronized (Message.class) {
            try (FileWriter writer = new FileWriter("messages.json", true)) {
                writer.write("{");
                writer.write("\"messageID\":\"" + escapeJSON(messageID) +
"\",");
                writer.write("\"messageHash\":\"" + escapeJSON(messageHash) +
"\",");
                writer.write("\"recipient\":\"" + escapeJSON(recipient) +
"\",");
                writer.write("\"messageText\":\"" + escapeJSON(messageText) +
"\"");
                writer.write("}");
                writer.write(System.lineSeparator());
                writer.flush();
                return true;
            } catch (IOException exception) {
                System.out.println("Error storing message: " +
exception.getMessage());
                return false;
            }
        }
    }

    private String escapeJSON(String text) {
        if (text == null) {
            return "";
        }

        return text.replace("\\", "\\\\")
```

```
                .replace("\"", "\\\"")
                .replace("\n", "\\n")
                .replace("\r", "\\r")
                .replace("\t", "\\t");
    }

    @Override
    public String toString() {
        return "Message{" +
                "messageID='" + messageID + '\'' +
                ", recipient='" + recipient + '\'' +
                ", messageNumber=" + messageNumber +
                ", messageHash='" + messageHash + '\'' +
                '}';
    }

    public String getMessageID() {
        return messageID;
    }

    public String getRecipient() {
        return recipient;
    }

    public String getMessageText() {
        return messageText;
    }

    public int getMessageNumber() {
        return messageNumber;
    }

    public String getMessageHash() {
        return messageHash;
    }

    public static List<Message> getSentMessages() {
        return Collections.unmodifiableList(sentMessages);
    }

    public static List<Message> getAllMessages() {
        return Collections.unmodifiableList(allMessages);
    }
}

public class Main {
    public static void main(String[] args) {
        Message message = new Message("+15551234567", "Hello!", 1);
```

```
        System.out.println(message.SentMessage(1));
        System.out.println("Total messages sent: " +
message.returnTotalMessages());
    }
}
```

2. QuickChat.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Message {

    private static final List<Message> sentMessages = new ArrayList<>();

    private final String recipient;
    private final String messageText;
    private final int messageNumber;
    private final String messageID;
    private final String messageHash;

    public Message(String recipient, String messageText, int messageNumber) {
        this.recipient = recipient == null ? "" : recipient.trim();
        this.messageText = messageText == null ? "" : messageText;
        this.messageNumber = messageNumber;
        this.messageID = String.format("%010d", Math.max(0, messageNumber));
        this.messageHash = createMessageHash();
    }

    public boolean checkMessageID() {
        return messageNumber > 0 && messageID.length() <= 10;
    }

    public String checkRecipientCell() {
        if (recipient.matches("\\+\\d{8,15}")) {
            return "Cell phone number successfully captured.";
        }
        return "Cell phone number is incorrectly formatted or does not include
an international code.";
    }

    public String validateMessage() {
        if (messageText.length() > 250) {
            return "Message exceeds 250 characters by " +
(messageText.length() - 250) + ", please reduce size.";
        }
        return "Message ready to send.";
    }

    public String SentMessage(int option) {
        if (option == 1) {
            sentMessages.add(this);
```

```
            return "Message successfully sent.";
        }
        if (option == 2) {
            return "Message disregarded.";
        }
        if (option == 3) {
            return "Message successfully stored.";
        }
        return "Invalid message option.";
    }

    public static List<Message> getSentMessages() {
        return Collections.unmodifiableList(sentMessages);
    }

    public String getMessageID() {
        return messageID;
    }

    public String getMessageHash() {
        return messageHash;
    }

    public String getRecipient() {
        return recipient;
    }

    public String getMessageText() {
        return messageText;
    }

    private String createMessageHash() {
        String[] words = messageText.trim().isEmpty()
                ? new String[0]
                : messageText.trim().split("\\s+");
        String firstWord = words.length > 0 ? words[0] : "";
        String lastWord = words.length > 0 ? words[words.length - 1] : "";
        String idPrefix = messageID.substring(0, Math.min(2,
messageID.length()));
        return (idPrefix + ":" + messageNumber + ":" + firstWord +
lastWord).toUpperCase();
    }
}
```

3. MessageTest.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.UUID;
import java.util.concurrent.atomic.AtomicInteger;

public class Message {

    private static final List<Message> sentMessages =
            Collections.synchronizedList(new ArrayList<>());
    private static final List<Message> allMessages =
            Collections.synchronizedList(new ArrayList<>());
    private static final AtomicInteger totalMessagesSent = new
AtomicInteger();

    private final String recipient;
    private final String messageText;
    private final int messageNumber;
    private final String messageID;
    private final String messageHash;

    public Message(String recipient, String messageText, int messageNumber) {
        this.recipient = recipient;
        this.messageText = messageText;
        this.messageNumber = messageNumber;
        this.messageID = UUID.randomUUID().toString().replace("-", "")
                .substring(0, 10);
        this.messageHash = createMessageHash();
        allMessages.add(this);
    }

    public boolean checkMessageID() {
        return messageID.length() == 10;
    }

    public String checkRecipientCell() {
        if (recipient != null && recipient.matches("\\+\\d{8,15}")) {
            return "Cell phone number successfully captured.";
        }
        return "Cell phone number is incorrectly formatted or does not contain
an international code. Please correct the number and try again.";
    }

    public String validateMessage() {
        if (messageText == null) {
```

```
            return "Message cannot be empty.";
        }
        if (messageText.length() > 250) {
            return "Message exceeds 250 characters by "
                    + (messageText.length() - 250)
                    + ", please reduce the size.";
        }
        return "Message ready to send.";
    }

    public String createMessageHash() {
        String firstWord = "";
        String lastWord = "";

        if (messageText != null && !messageText.trim().isEmpty()) {
            String[] words = messageText.trim().split("\\s+");
            firstWord = cleanWord(words[0]);
            lastWord = cleanWord(words[words.length - 1]);
        }

        return messageID.substring(0, 2).toUpperCase()
                + ":" + messageNumber + ":" + firstWord + lastWord;
    }

    private String cleanWord(String word) {
        return word.replaceAll("[^\\p{L}\\p{N}]", "").toUpperCase();
    }

    public String SentMessage(int action) {
        if (!"Cell phone number successfully
captured.".equals(checkRecipientCell())
                || !"Message ready to send.".equals(validateMessage())) {
            return "Message could not be processed.";
        }

        if (action == 1) {
            sentMessages.add(this);
            totalMessagesSent.incrementAndGet();
            return "Message successfully sent.";
        }
        if (action == 2) {
            boolean removed = sentMessages.remove(this);
            if (removed) {
                totalMessagesSent.decrementAndGet();
                return "Message successfully disregarded.";
            }
            return "Message not found in sent messages.";
        }
```

```
        if (action == 3) {
            return "Message successfully stored.";
        }
        return "Invalid message action.";
    }

    public String getMessageID() {
        return messageID;
    }

    public String getMessageHash() {
        return messageHash;
    }

    public String getRecipient() {
        return recipient;
    }

    public String getMessageText() {
        return messageText;
    }

    public static List<Message> getSentMessages() {
        synchronized (sentMessages) {
            return new ArrayList<>(sentMessages);
        }
    }
}
```

4. Pom.xml

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Message {

    private static final List<Message> SENT_MESSAGES = new ArrayList<>();
    private static final List<Message> STORED_MESSAGES = new ArrayList<>();

    private final String recipient;
    private final String messageText;
    private final int messageNumber;
    private final String messageID;
    private final String messageHash;

    public Message(String recipient, String messageText, int messageNumber) {
        this.recipient = recipient == null ? "" : recipient.trim();
        this.messageText = messageText == null ? "" : messageText;
        this.messageNumber = messageNumber;
        this.messageID = String.valueOf(messageNumber);
        this.messageHash = createMessageHash();
    }

    public boolean checkMessageID() {
        return messageID.length() <= 10;
    }

    public String checkRecipientCell() {
        if (recipient.matches("\\+\\d{8,15}")) {
            return "Cell phone number successfully captured.";
        }
        return "Cell phone number is incorrectly formatted or does not contain
an international code.";
    }

    public String validateMessage() {
        if (messageText.isEmpty()) {
            return "Message cannot be empty.";
        }
        if (messageText.length() > 250) {
            return "Message exceeds 250 characters by " +
(messageText.length() - 250) + ", please reduce size.";
        }
        return "Message ready to send.";
    }

    public String SentMessage(int choice) {
```

```
        if (choice == 1) {
            SENT_MESSAGES.add(this);
            return "Message successfully sent.";
        }
        if (choice == 3) {
            STORED_MESSAGES.add(this);
            return "Message successfully stored.";
        }
        return "Message disregarded.";
    }

    public static List<Message> getSentMessages() {
        return Collections.unmodifiableList(SENT_MESSAGES);
    }

    public static List<Message> getStoredMessages() {
        return Collections.unmodifiableList(STORED_MESSAGES);
    }

    public String getMessageID() {
        return messageID;
    }

    public String getMessageHash() {
        return messageHash;
    }

    public String getRecipient() {
        return recipient;
    }

    public String getMessageText() {
        return messageText;
    }

    private String createMessageHash() {
        String[] words = messageText.trim().split("\\s+");
        String firstWord = words.length == 0 || words[0].isEmpty() ? "" :
words[0];
        String lastWord = words.length == 0 || words[words.length -
1].isEmpty()
                ? "" : words[words.length - 1];
        String idPrefix = messageID.length() < 2 ? messageID :
messageID.substring(0, 2);
        return (idPrefix + ":" + messageNumber + ":" + firstWord +
lastWord).toUpperCase();
    }
}
```